

LSA Service Booking API - Detailed Usage Guide

This document provides a clear, step-by-step guide on how to test and interact with your backend API. It assumes you have your development server running and your `seed_db.py` data loaded.

1. Getting Started

First, make sure your Django development server is running in your virtual environment:

```
# In your terminal (make sure (venv) is active)
python manage.py runserver
```

Next, open the built-in interactive API documentation (Swagger UI) in your web browser:

<http://127.0.0.1:8000/api/docs/swagger/>

2. Authentication (Login)

Almost all endpoints in this project require you to be authenticated using a JWT (JSON Web Token). We need to exchange your username (`vannuru`) and password (`swamy07`) for a token.

Step 2.1: Generate the Token

1. In the Swagger UI, scroll down to the **auth** section.
2. Click on the row for `POST /api/v1/auth/token/` to expand it.
3. Click the **Try it out** button on the right.
4. In the **Request body** text area, enter your credentials:

```
{
  "username": "vannuru",
  "password": "swamy07"
}
```

- Click the large blue **Execute** button.
- Scroll down to the **Responses** section. You should see a 200 status code and a response body that looks like this:

```
{
  "refresh": "eyJhbGciOiJIUz...",
  "access": "eyJhbGciOiJIUzI1NiIsInR..."
}
```

7. **Copy the long text inside the quotes of the "access" field.** Make sure you do NOT copy the quotes themselves.

Step 2.2: Authorize Swagger UI

Now that we have the access token, we must tell Swagger to attach it to all future requests.

1. Scroll to the very top of the Swagger webpage.
2. Click the green **Authorize** button (with a padlock icon).
3. A modal will pop up. In the **Value** field under **jwtAuth (http, Bearer)**, just paste your raw token (e.g., eyJhbGciOiJIUzI1NiIsInR...). *Do NOT type the word "Bearer".*
4. Click **Authorize**, then click **Close**.

3. Searching for LSAs

Now that you are logged in, let's search for available Learning Support Assistants (LSAs).

1. Scroll down to the **Isas** section in Swagger.
2. Expand GET /api/v1/lsas/search/ and click **Try it out**.
3. You will **see fields like** skill, start_time, end_time, ordering, etc.
 - o **To see all LSAs:** Leave everything blank and just click **Execute**.
 - o **To filter by skill:** Type `autism` or `math` into the `skill` field and click **Execute**.
4. Check the **Responses** section. You will get a JSON list of LSAs. Note down the "id" of an LSA you want to book!

4. Making a Booking

Let's book an LSA for the Parent (ID: 2).

1. Scroll to the **bookings** section.
2. Expand `POST /api/v1/bookings/` and click **Try it out**.
3. The **server** will automatically pre-populate the text box with a perfectly valid JSON payload (dates scheduled exactly for tomorrow!). You just need to change the `parent_id` and `lsa_id` to match your data.

```
{
  "parent_id": 2,
  "lsa_id": 3,
  "session_start": "2026-08-11T10:00:00Z",
  "session_end": "2026-08-11T12:00:00Z"
}
```

4. Click **Execute**.

Expected Result

If the LSA is available, you will receive a 201 Created response looking something like this:

```
{
  "id": 1,
  "parent_id": 2,
  "lsa_id": 3,
  "status": "pending",
  "payment": {
    "id": 1,
    "razorpay_order_id": "order_XXXXXXX",
    "amount": 300000,
    "currency": "INR",
    "status": "created"
  }
}
```

Copy the "razorpay_order_id" from the payment block. You need it for the final step.

5. Simulating the Payment Webhook

Because this is a backend-only project without a frontend checkout screen, we need to simulate Razorpay confirming the payment.

1. Open a new terminal and make sure you are in your project folder (A:\Python Backend Project) with (venv) active.
2. Run the `simulate_webhook.py` script and pass your Order ID from Step 4 as an argument:

```
python simulate_webhook.py order_XXXXXXX
```

3. The script will securely sign a JSON payload mimicking Razorpay and POST it to your server.
4. You should see a success message (Status Code: 200)!
5. The payment and booking statuses in your database have now been updated from PENDING/CREATED to COMPLETED!